

YaysApp

Module Development Plan & Timeline

INDEXX.AI · Confidential Product Planning
Revision v3.0 (module-based) · 2026-08-04

YaysApp — Module-Based Development Plan (v3.0)

Supersedes the feature-milestone plan in `aychat_ai_development_milestones.md` (v2.1). That document organised work as ten sequential **feature milestones** that front-loaded a complete mock frontend before any real backend. This revision reorganises the same scope into ***capability modules, cuts to a functional MVP, and reorders so that real-time messaging between real users ships first***.

Naming: the product is **YaysApp** (formerly "Yay-chat" / "YayChat"). Use "YaysApp" in all human-facing text. On-disk identifiers are unchanged: directory `YayChat/`, remote `daniyal198/yaychat`, package `yaychat-mobile` — do not rename them.

Sources of truth (unchanged): product = `yaychat_unified_product_vision_prd.md` (PRD v2.1); integration/technical = `YayChat_Platform_Integration_Architecture_v2.1`. Where anything conflicts, those v2.1 documents win.

1. What changed and why

	v2.1 milestone plan	v3.0 module plan (this doc)
Unit of work	10 sequential feature milestones	Capability modules that evolve independently
First deliverable	Entire mock frontend (Milestone 1)	Real-time chat between real users
Backend	Milestone 2, after the whole mock UI	Thin slice brought forward to unblock chat
MVP	Broad (chat + communities + AI + Mini App + deep-link + growth)	Narrow: the smallest product that delivers working real-time messaging
Everything else	In the linear sequence	Explicitly deferred behind the MVP cut line

highest-value capability is messaging that actually works between two real people on two real devices. Building a complete mock frontend first proves **layout**, not **product**. So we invert the order: stand up only the foundation chat needs, ship real messaging, then grow outward module by module.

2. Global rules (carried forward, still binding)

These are hard constraints from the PRD/architecture and apply to **every** module:

- **YaysApp is a hub, not a super app.** It never contains — in any form, including "temporary" or

"demo" builds that reach users — wallets, key storage, seed phrases, send/receive/swap, buying or selling assets, Power Mining / Mining Station purchase, executing mining/Alchemy/rewarded-ad views, crediting/debiting Nuggets/BTCY/points, placing/settling predictions, checkout, or KYC for financial purposes. **If a requirement seems to need one of these, it is a deep link, not a feature.**

- **No mini-program runtime.** Mini Apps render read-only dashboards from manifests; they never

execute product code or sandbox third-party JS.

- **Original identity.** Not a renamed Bitcoin Yay, not a visual clone of WeChat/WhatsApp/Telegram.
- **Reuse chat logic only.** The `bitcoin-yay-mobile` repo may be inspected for chat-related code

(message models, sockets, sync, media, receipts, presence, pagination) and nothing else.

- **Never present mock data as real** — especially never a real-looking balance from mock data.
 - **Deep-link first, read-only API second, native execution never.** The hub holds no financial state.
-

3. MVP definition (the cut line)

time — 1:1 and in groups — reliably, with media and voice notes, delivery/read state, and push.*

Everything required to make that true is **MVP-core**. Everything else is **Deferred** (built later, in module order). The MVP-core modules are **M0, M1, M2**, plus the thin slice of **M6** (push + crash/analytics) that chat needs to be trustworthy.

4. Module catalogue

Each module lists: priority, objective, scope, dependencies, and the acceptance criteria that mean "done." Priority is one of **MVP-core**, **MVP-adjacent** (next, right after core), or **Deferred**.

M0 — App Foundation & Design System · *MVP-core (thin)*

needed to host chat — no more.

Profile — but only Chats wired for MVP), state management, service-adaptor seam (mock↔real without touching screens), original design tokens + the core component set (buttons, inputs, lists, message bubbles, composer, sheets, toasts, skeletons, empty/error/offline states). Product-specific components (Mini App Card, Dashboard Card, ActionButton, Cross-Promotion Slot, External Link Warning) are **specified but stubbed** until their modules arrive.

in place; a screen can switch from mock to real service by swapping an adapter, not UI.

M1 — Identity & Unified Auth · *MVP-core (thin slice)*

email/phone verification, profile + avatar, device registration. **Account-linking foundation** (linked product accounts, read-only scopes) is *scaffolded* (data model + settings screen) but full unified OAuth linking against central Indexx identity is **deferred to M7-adjacent**.

real APIs; profiles persist; authorisation enforced.

M2 — Chat / Real-Time Messaging · *MVP-core · HIGHEST PRIORITY*

history with pagination and stable ordering; send retries + idempotency; delivery & read receipts; typing indicators; presence; replies, reactions, mentions, forwarding, deletion/recall, pinned messages; media + file upload; voice notes; in-conversation and global search; offline queue + sync; multi-device; block/report; push notifications that open the correct conversation. Indexx links render as branded **Action Cards**; other links get the **External Link Warning** (rendering only — no handoff yet).

is reusable vs must be rebuilt cleanly.

reconnects; duplicate sends prevented; offline sync works; media/voice upload securely; a push opens the right conversation; block/report work; critical flows have automated tests.

M6 — Notifications, Analytics & Admin · *thin slice MVP-core, rest Deferred*

crash reporting, and a minimal analytics/event pipeline. **Deferred:** full admin portal, moderation dashboards, rich analytics warehouse. captured.

M3 — Communities & Announcements · *MVP-adjacent*

enforcement; community chat, polls, events, rules; official **verified accounts** per product with a publishing-approval workflow; broadcast announcements with scheduling, audience/region targeting, read analytics (one primary action link each — minted through the link service, mocked until M4); reporting, banning, moderation queues, impersonation detection.

post official announcements; announcements schedulable, targetable, measurable; moderation reports enter a workflow.

M5 — AI Assistant & Support Desk · *MVP-adjacent*

and suggested prompts; explicit consent before private chat content reaches a model; financial/legal/ medical disclaimers, never personalised investment advice, never guaranteed-returns framing; AI inside chats/communities; support desk with AI first line, ticket creation, and escalation to a human queue carrying product/Mini-App context. Ecosystem answers end with the correct **Action Card** (mocked link until M4).

support tickets escalate with context.

M4 — Mini App Hub, Bitcoin Yay Mini App & Deep-Link / Attribution · *MVP-adjacent (strategic)*

tracked deep-link handoffs, proven end-to-end with Bitcoin Yay. **No wallet is built** (this replaces the old "wallet milestone").

dashboard renderer for the fixed card types (amount, progress, status, count, list); region/age/link gating; framework guarantee that no card or action can write. Bitcoin Yay read adapter (Nuggets, token balance, mining/alchemy progress, station status, referrals, notifications, eligibility) with caching, rate limits, circuit breakers, stale-with-timestamp degradation. Deep-link service: route registry, signed single-use expiring handoff tokens, destination allowlist, Universal/App Links + web routes + store fallback + deferred install resolution, handoff token exchange (arrive signed-in), outcome recording, per-route/per-campaign kill switch. Attribution: authenticated inbound completion events, schema validation, idempotency, matching to handoff/Mini-App/campaign/surface/referrer, fraud signals.

Bitcoin Yay dashboard matches the product's own values with source + freshness; no service can write a balance/asset/reward (verified by review + test); a confirmed action is traceable to its handoff; duplicate events don't double-count; deep-link routes tested incl. not-installed, web, and expired paths.

M7 — Growth: Referrals, Campaigns, Cross-Promotion, Academy · *Deferred*

campaign cards across surfaces; cross-promotion service + per-Mini-App slot; multi-product quests driven by verified events; daily check-in/streaks; referral centre (per-product code, tracked sharing, deferred referral links, leaderboards); Index Academy (courses, lessons, quizzes, lesson-ending Action Cards); frequency capping; A/B variants; anti-abuse. Rewards are ****defined, funded, and credited by the owning product — never by YaysApp****.

code change; cross-promo records conversions; quest progress updates only from verified events; referral links survive install and attribute correctly.

M8 — Remaining Mini Apps, Merchant & Advertising · *Deferred*

(region-gated) Mini Apps — each added by manifest + summary endpoint, no core change. Merchant accounts/verification/promoted posts; sponsored communities/channels/posts with clear labelling; merchant analytics/billing/fraud prevention. ****Rewarded/banner ad inventory stays inside the product apps — YaysApp promotes and links, never serves ads.****

invisible outside approved regions; merchants verifiable; ads labelled and auditable.

M9 — Production Hardening & Launch · *Deferred (pre-public-release gate)*

tests^{**} — forged links, replayed tokens, tampered payloads, open-redirect — and manifest validation), load/stress testing, accessibility, localisation readiness, crash/metrics/alerts, incident response, backup/restore + DR tests, store assets + per-market policy review (a chat app promoting token/mining activity gets extra scrutiny), financial-promotion copy review, release signing, staged rollout, rollback.

platform; malformed/unregistered manifests rejected; performance targets met; rollback path exists and release approval recorded.

5. Deferred / removed from MVP (explicit)

To keep the MVP functional and small, the following are **postponed** (order preserved above), not built for first release:

- Communities & Announcements (M3), AI & Support (M5) — valuable, but not required to prove

real-time messaging. Bring up immediately **after** the chat MVP.

- Mini App Hub, Bitcoin Yay Mini App, Deep-Link & Attribution, full unified linking (M4) — the

strategic core of the ***hub***, but heavy; sequence it right after the communication layer is solid.

- Growth/referrals/campaigns/academy (M7), remaining Mini Apps + merchant/ads (M8).
- Voice/video calls, and any third-party mini-program runtime — **out of scope by design**, not a gap.

6. Per-module output discipline (unchanged from v2.1 §16)

For each module: start with objective, code/systems inspected, assumptions, open decisions, approach, task breakdown, files to change, risks, acceptance criteria. During work, keep scope to the active module, keep the app runnable, add tests alongside critical behaviour, update docs, and ****flag immediately**** if a requirement appears to need a wallet, balance write, ad serving, or runtime — that is a scope error, not a task. End with completed/remaining items, files changed, tests + results, known limitations, open decisions, security concerns, and confirmation that acceptance criteria were met. **Do not start the next module until the current one meets its acceptance criteria.**

7. Critical-path note for the autonomous workflow

The chat MVP requires **real backend work** (sockets, delivery, media, push). The local backend `npm install` is currently broken, and the nightly autonomous workflow is scoped to `mobile/` only.

bot can progress M0 (design system, shell, chat UI on mock adapters) and the chat audit, but cannot land real-time messaging. See the

companion `yaysapp_module_timeline.md` for sequencing.

YaysApp — Development Timeline & Implementation Order

Companion to `yaysapp_module_development_plan.md` . That document defines *what* each module is; this one sets the **deadline** for each and the order to build them. Chat is first: the MVP is real-time messaging between real users.

Cadence: every module has a hard **2–3 day** deadline — deliberately aggressive targets that force tight scope and daily shipping, not classic effort estimates.

Trimmed scope: the two Deferred modules — ****M7 Growth (referrals/campaigns/cross-promotion/academy) and M8 Remaining Mini Apps + Merchant/Advertising — are cut from this timeline**** and parked as post-launch work. What remains is the smallest path from nothing → a real chat product → the hub differentiator → launch.

Per-module deadlines (trimmed)

Module	Priority	Deadline	Start after
M0 App Foundation & Design System	MVP-core	2 days	—
M1 Identity & Unified Auth *(thin slice)*	MVP-core	2 days	M0 (overlaps)
M2 Chat / Real-Time Messaging	MVP-core · #1	3 days	M1
M6 Notifications/Analytics *(thin slice)*	MVP-core	1 day	M1 (overlaps M2)
— MVP CUT LINE — Chat MVP live		~day 6	
M3 Communities & Announcements	MVP-adjacent	2 days	M2
M5 AI Assistant & Support Desk	MVP-adjacent	2 days	M2 (overlaps M3)
M4 Mini App Hub + BTCY + Deep-Link/Attribution	MVP-adjacent	3 days	M1, M6
M9 Production Hardening & Launch	Launch gate	2 days	all above
M7 Growth	*cut — post-launch*	—	—
M8 Remaining Mini Apps + Merchant/Ads	*cut — post-launch*	—	—
Total (trimmed)		~14 days	~2 weeks with overlap

modules overlapping where their bottlenecks differ.

Recommended implementation order (day-by-day)

Phase 0 — Foundation (days 1–2) — MVP-core

M0 App Foundation & Design System (2d)

M1 Identity & Auth (thin) (2d)

design + RN in parallel

backend starts here (overlaps M0)

Phase 1 — THE CHAT MVP (days 3–6) — MVP-core · the goal

M2 Chat / Real-Time Messaging (3d)

M6 Notifications (thin: push/crash) (1d)

← highest priority

← runs alongside M2

► SHIP by day 6: two real users message in real time (1:1 + group, media, voice, receipts, push)

Phase 2 — Communication layer (days 7–10) — MVP-adjacent

M3 Communities & Announcements (2d)

M5 AI Assistant & Support (2d)

overlaps M3 (different bottlenecks)

Phase 3 — Hub differentiator (days 9–12) — MVP-adjacent, strategic

M4 Mini App Hub + Bitcoin Yay + Deep-Link/Attribution + full unified linking (3d)

► SHIP: read-only dashboard + one-tap deep-link handoff into Bitcoin Yay, attributed

Phase 4 — Launch gate (days 13–14) — pre-release

M9 Production Hardening & Launch (2d)

Cut from this timeline (post-launch): M7 Growth · M8 Remaining Mini Apps + Merchant/Ads

Day ranges overlap where modules have different bottleneck disciplines (e.g. M5's AI/backend work runs while M3's moderation UI is in mobile hands).

Critical path

1. **Backend build must be unblocked on day 1.** The local backend `npm install` is broken and the nightly autonomous workflow is scoped to `mobile/` only. ****Real-time messaging cannot land until the backend is buildable**** — with a 3-day M2 deadline, fix this on day 1 or the chat MVP slips. 2. **M2 (Chat) is the long pole** even at 3 days — sockets, media, offline sync, and push all land in one window. Spike the real-time transport and media upload on day 1 to protect the deadline. 3. **M0/M1 overlap**; M6-thin overlaps M2; all of Phase 2 depends on M1+M2.

What the nightly autonomous workflow can do now vs. later

- **Now (mobile-only, backend blocked):** M0 (design system, app shell, navigation), the chat UI on mock adapters, the `bitcoin-yay-mobile` chat audit, and mobile-side tests.
- **After the backend build is fixed:** real M1 auth and M2 real-time messaging — the MVP itself.

Checkpoints / demos

Checkpoint	Due	Proves
C0 Shell + design system runs	day 2	Original-branded app, navigation, no mock-as-real
C1 Real login between two devices	day 4	Real accounts + sessions
C2 — MVP Real-time chat, 2 real users	day 6	1:1 + group, media, voice, receipts, push
C3 Communities + AI live	day 10	Gather + broadcast + assistant
C4 Bitcoin Yay Mini App + deep link	day 12	Hub thesis: chat → attributed action
C5 Launch-ready	day 14	Hardened, reviewed, staged rollout

These deadlines are intentionally tight. Protect **C2 (day 6)** above all — the chat MVP. If the backend build isn't unblocked on day 1, re-baseline C2 first; every later date hangs off it.